KitPri v4: Cooking-Sound Detection for SmartThings Edge Devices via Cross-Architecture Knowledge Distillation and INT8 Quantization

Samsung PRISM — Worklet 25ST31BMS

Ayush Pandey
B.M.S. College of Engineering, Bengaluru
`github.com/ayushpandey0201/kitpribot`

July 2026

Abstract

We present KitPri v4, a complete, reproducible pipeline that detects cooking activity from 10-second audio clips and fits the resource envelope of a Samsung SmartThings edge device (∼60 MB RAM budget, CPU-only inference). A 329 MiB Audio Spectrogram Transformer (AST) teacher (86.2 M parameters, test F1 0.8129) is distilled into a 2.23 M-parameter MobileNetV2 student (8.49 MB, test F1 0.7318 after validation-only threshold tuning), which is then compressed by static post-training INT8 quantization to a **2.80 MB** TorchScript artifact (test F1 0.6910) — an end-to-end compression of more than 118× with a 2.25× CPU speed-up (57.3 → 25.5 ms per clip). Training data is a purpose-built corpus of 7,200 synthetic mixtures of 2–4 layered sound sources, including an adversarially hard overlap regime in which the cooking source can be up to 5 dB *quieter* than a competing sound. The evaluated model is served end-to-end through a single shared `Predictor` code path — CLI, batch evaluation, and a public Telegram bot hosted 24/7 on an Oracle Cloud Always-Free VM under systemd supervision at zero recurring cost. We report methodology, results, deployment architecture, failure modes, and open limitations in full.

1 Introduction

Ambient acoustic sensing is a natural fit for smart-home platforms: cooking produces a rich, sustained acoustic signature (sizzling, boiling, chopping, exhaust hum) that a microphone-equipped hub can observe passively. A reliable Cooking/Not-Cooking signal enables downstream SmartThings automations — ventilation control, safety timers, presence inference — without cameras or wearables.

The engineering obstacle is not accuracy in isolation; it is accuracy *under an edge budget*. State-of-the-art audio classifiers are transformers with hundreds of megabytes of weights, while the deployment target offers roughly 60 MB of RAM, no accelerator, and a CPU that must share time with the rest of the hub's workload. KitPri v4 addresses this gap with a three-stage compression pipeline (Figure 1) and treats everything around the model — dataset realism, preprocessing parity, threshold governance, packaging, and hosting — as first-class engineering concerns.

Contributions.

1. A synthetic-mixture dataset methodology (7,200 clips) in which *every* clip layers 2–4 sources at varying relative gains, deliberately including a hard overlap regime (Section 4).

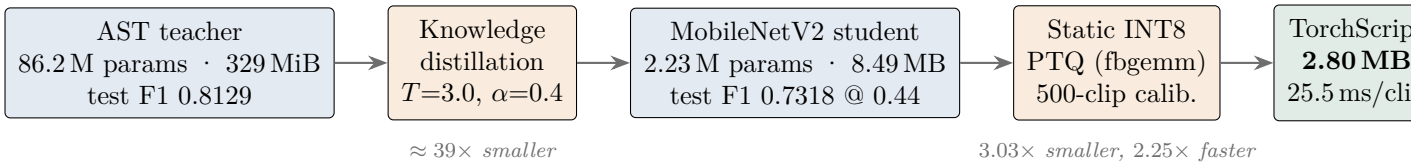

Figure 1: The KitPri v4 compression pipeline: $> 118\times$ end-to-end size reduction from teacher checkpoint to deployed artifact.[1]

2. A benchmarked architecture selection: six EfficientNet-B0 variants plateau on v4 data while AST exceeds them within one epoch (Section 5.1).
3. A cross-architecture distillation recipe (transformer → CNN) with an explicit, documented loss convention (Section 5.2).
4. A quantization study demonstrating why dynamic PTQ fails for CNN-style models and static PTQ succeeds, with measured size/latency/accuracy trade-offs (Section 5.4).
5. A deployment architecture with strict train/serve parity through one shared `Predictor`, an x86-only-artifact guard, Docker packaging, and a zero-cost, always-on cloud demonstration bot (Section 7).
6. An honest limitations register, including error concentration on overlapped audio and an un-retuned INT8 threshold (Section 8).

2 Background and Related Work

Audio Spectrogram Transformer (AST) [1] applies a ViT-style patch attention mechanism directly to log-mel spectrograms and transfers well from AudioSet pretraining; we use the HuggingFace `ASTForAudioClassification` implementation. **MobileNetV2** [2] remains a strong efficiency baseline built on inverted residuals and depthwise separable convolutions; we use the `timm` `mobilenetv2_100` variant. **Knowledge distillation** [3] transfers the teacher's softened output distribution to a smaller student; the classic T^2 gradient-scale correction applies when mixing soft and hard losses. **Post-training quantization** in PyTorch offers dynamic (weights-only) and static (weights + activations, calibration-based) modes; the latter is required when convolutional activation ranges must be fixed ahead of time [5]. Negative (non-cooking) source audio derives from **ESC-50** [4] (CC BY-NC 3.0, which constrains commercial redistribution of the derived dataset).

3 Problem Formulation and Constraints

Given a 10-second audio clip x, predict $y \in \{\text{Not-Cooking} = 0,\ \text{Cooking} = 1\}$. The model emits a single logit z; the served decision is $\hat{y} = \mathbb{1}[\sigma(z) \geq \tau]$ with threshold τ selected on validation data only (Section 5.3).

4 Dataset (v4)

4.1 Why synthesis, and lessons from v1–v2

Earlier iterations exposed two failure classes that motivated v4's design. The v1 build suffered label corruption discovered during a June review; v2 used clean, single-source clips and trained

[1] Computed conservatively as 329/2.80. Converting the teacher checkpoint strictly to MB (329 MiB ≈ 345 MB) yields 123×.

Constraint	Value
Target device	Samsung SmartThings hub-class edge device
Memory envelope	∼60 MB RAM for the entire inference stack
Compute	CPU only (no GPU/NPU assumed)
Input contract	any container/rate/length; normalized in preprocessing
Serving parity	training, evaluation, and demo must share one code path
Cost of demo infrastructure	$0 recurring (Section 7.4)

Table 1: Deployment constraints that shaped the design.

models that missed quiet cues (e.g. electrical hums) entirely — clean data taught an unrealistically easy decision boundary. v4 therefore synthesizes *every* clip as a mixture of **2–4 layered sources at varying relative volumes**, seeded (`seed=1337`) for exact reproducibility.

Total clips	7,200 (10.0 s each)
Split	6,300 train / 450 val / 450 test
Test balance	exactly 225 COOKING / 225 NOT-COOKING
COOKING sources	10 organised subtypes (sizzle, boil, chop, …)
NOT-COOKING sources	1,160-clip pool derived from ESC-50
Hard regime (type C)	17% of clips: cooking + competing sound, SNR $\in [-5, +20]$ dB
Distribution	Kaggle (`ayushalia/kitpri-v4-dataset`); regenerable via script

Table 2: v4 corpus. In type-C clips the cooking source can be up to 5 dB *quieter* than the competing non-cooking source.

The asymmetry of the two classes is deliberate and consequential: COOKING is a coherent category with 10 curated subtypes, whereas NOT-COOKING is an open-world pool of 1,160 heterogeneous sounds. Section 6.3 shows this asymmetry surfacing in the error structure.

4.2 Preprocessing (identical at train and serve time)

Sample rate / channels	32,000 Hz, downmixed to mono
Duration	pad or truncate to exactly 10.0 s
Mel filterbank	128 bins, n_{fft}=1024, hop 512
Amplitude scaling	`AmplitudeToDB(top_db=80)`
Normalization	per-clip $(x - \mu_x)/(\sigma_x + 10^{-6})$
Channel expansion	spectrogram repeated to 3 channels
Model input	$(1, 3, 128, 626)$

Table 3: The single preprocessing contract used by trainer, evaluator, CLI, and bot.

Porting pitfall (empirically verified). There is *no* resize to 224×224 and *no* fixed ImageNet normalization. The spectrogram keeps its natural 128×626 geometry and is standardized by its own statistics. Introducing a `Resize` or a dataset-constant `Normalize` degrades accuracy silently — no shape error is raised, the distribution is simply wrong.

5 Method

5.1 Architecture selection and the teacher

EfficientNet-B0 — adequate on the easier v2 data — was evaluated on v4 in six configurations spanning freezing strategy, learning rate, dropout, augmentation, and mixup. All six landed in one narrow performance band, indicating an *architecture* ceiling rather than a tuning deficiency:

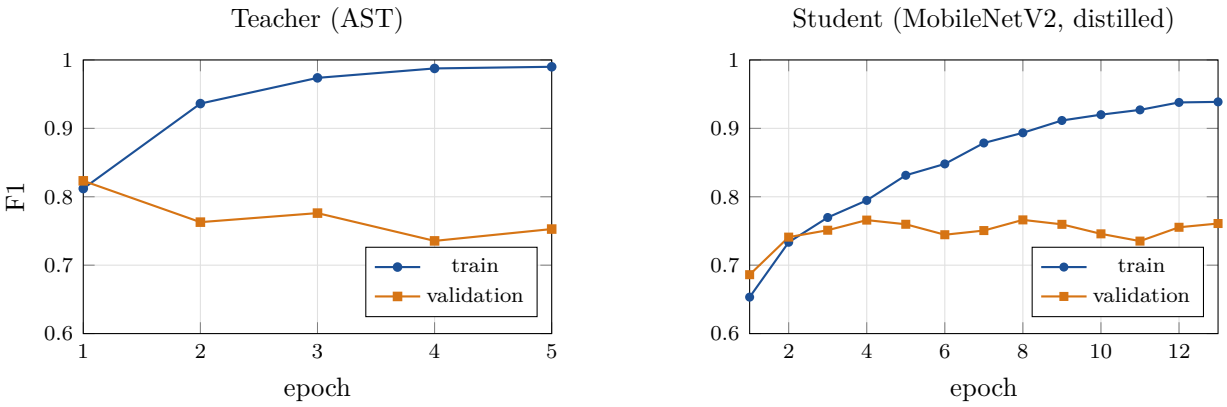

Figure 2: Training dynamics (F1 per epoch, from `training_log.csv`). Left: the teacher peaks at epoch 1 and overfits immediately — it is data-limited. Right: the distilled student improves steadily to its best epoch (8) with a stable validation plateau.

image-style convolutional filters are a poor prior for spectro-temporal structure under heavy source overlap. AST, using attention over spectrogram patches, surpassed every EfficientNet run *within its first epoch* and was adopted as the teacher.

	Teacher (AST)	**Student (distill)**	**Quantization**
Max epochs / batch	20 / 16	30 / 16	—
LR / weight decay	10^{-4} / 0.01	$3{\cdot}10^{-4}$ / 10^{-4}	—
Schedule	cosine	cosine	—
Early stop (patience)	val F1 (5)	val F1 (6)	—
Outcome	best epoch 1, stop @ 6	best epoch 8, stop @ 14	500 calib. clips

Table 4: Training configuration (verified from the original training scripts in `training/` and `configs/experiments/*.yaml`; outcomes from `run_summary.json`).

The teacher's validation F1 peaks at **epoch 1** (0.8233) and declines thereafter while train F1 climbs beyond 0.98 (Figure 2, left): with 86.2 M parameters against 6,300 training clips, the teacher is *data*-limited, not capacity-limited. This is the central argument that further gains should come from data diversity and augmentation rather than larger models.

5.2 Cross-architecture distillation

The student is `timm mobilenetv2_100` (2,225,153 parameters) receiving the same $(3, 128, 626)$ input. With student logit z_s, teacher logit z_t, hard label y, temperature T=3.0 and α=0.4, the loss implemented in `src/kitpri/training/distill.py` is

$$\mathcal{L} \;=\; \alpha \underbrace{\mathrm{BCE}(\sigma(z_s),\, y)}_{\text{hard}} \;+\; (1-\alpha)\, T^2 \underbrace{\mathrm{BCE}(\sigma(z_s/T),\, \sigma(z_t/T))}_{\text{soft}}, \tag{1}$$

i.e. α *weights the hard-label term*; the T^2 factor restores gradient magnitude for the temperature-softened term [3]. We state this convention explicitly because α's role is inconsistent across the literature; it is verified directly against the original training script (`training/distill_mobilenet.py`). Training-time SpecAugment (frequency masking up to 27 mel bins, time masking up to 80 frames) regularizes the student; no masking is applied at evaluation.

The student's generalization gap at its best epoch (train−val F1 = 0.127) is markedly healthier than the teacher's post-epoch-1 trajectory, and val F1 remains on a plateau ($\approx$ 0.74–0.77) rather than collapsing (Figure 2, right).

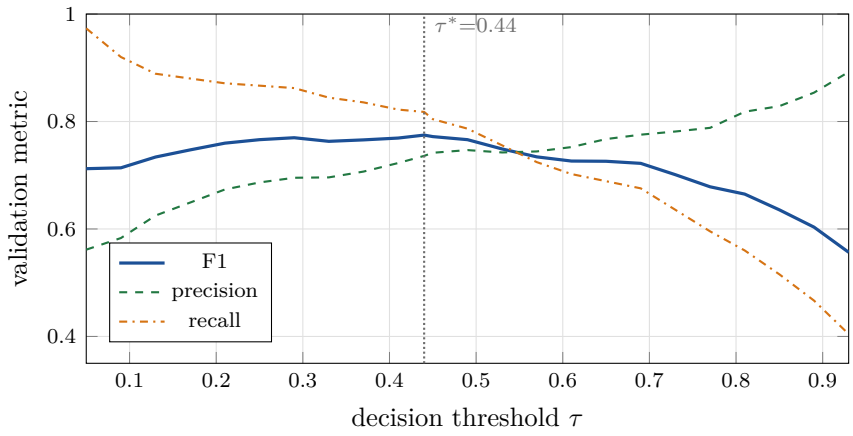

Figure 3: Validation threshold sweep for the FP32 student (`student_threshold_sweep.csv`, downsampled for legibility).

5.3 Decision-threshold governance

The operating threshold was swept **on the validation set only**; the test set was never consulted. The sweep (Figure 3) selects $\tau^* = 0.44$ (val F1 0.7747), lifting test F1 from 0.7226 (at the 0.50 default) to **0.7318**. A documented low-false-alarm alternative exists at $\tau = 0.87$ (val FPR 0.098) but collapses recall to 0.493 and is rejected for the primary use case.

5.4 INT8 static post-training quantization

An initial attempt with *dynamic* PTQ broke the model outright: dynamic mode quantizes weights only and leaves activation ranges to be computed on the fly, which is inappropriate for convolutional feature maps whose ranges must be fixed and calibrated. The shipped artifact instead uses **static PTQ**: observers inserted, **500 real training clips** run through the network for calibration, weights and activations converted to INT8, and the result exported as TorchScript (2.80 MB).

Two engine-level facts dominate deployment (Section 7.2): the artifact was packed with the `fbgemm` (x86) engine, and under ARM's `qnnpack` engine it loads but emits collapsed, meaningless probabilities ($\sim$0.01 for all inputs; verified empirically). The INT8 output grid is also coarse near the decision boundary (≈ 0.216 per logit step, i.e. $\approx$5 percentage points of probability), which interacts with the threshold discussion in Section 8.

6 Results

6.1 Headline metrics

Model	τ	Size	F1	Acc	Prec	Rec	AUC	ms/clip
AST teacher (86.2 M)	0.50	$\sim$329 MiB	**0.8129**	0.8200	0.8462	0.7822	0.8976	—
Student FP32 (2.23 M)	0.50	8.49 MB	0.7226	0.7356	0.7598	0.6889	0.8018	57.3
Student FP32, tuned	0.44	8.49 MB	**0.7318**	0.7378	0.7488	0.7156	0.8018	57.3
Student INT8 (static PTQ)	0.50	**2.80 MB**	0.6910	0.7178	0.7634	0.6311	—	**25.5**

Table 5: Test-set results (450 balanced clips). Latency is single-thread x86 CPU, measured over the full test pass (`quantization_report.json`).

Distillation retains 89.0% of the teacher's F1 in 2.6% of its parameters; quantization then trades a further 0.032 F1 for a 67.0% size cut and a 2.25× latency win. The INT8 artifact

preserves *precision* (0.763, the best of all student rows) while giving up recall — consistent with quantization eroding low-confidence positive probabilities near the boundary rather than corrupting confident detections.

6.2 Confusion structure

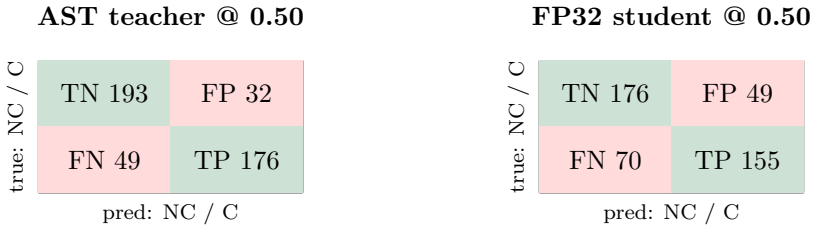

Figure 4: Test confusion matrices (NC = Not-Cooking, C = Cooking; 225 clips per class). Both models err more by *missing* cooking than by false alarms.

6.3 Where the errors live

Errors concentrate almost entirely in the by-design hard regime: on type-C clips (cooking overlapped with a competing sound at SNR down to −5 dB; 17% of the corpus) accuracy drops to roughly **39–50%**, versus **75–78%** on clean, non-overlapping clips. Two structural factors compound this: the Not-Cooking class is an unstructured 1,160-sound pool (harder to characterize than the 10 curated cooking subtypes), and the teacher itself is data-limited (Section 5.1), capping the soft targets the student can learn from.

7 Deployment

7.1 One `Predictor`, everywhere

All consumers — `inference/predict.py`, batch evaluation, and the Telegram bot — import the same `kitpri.inference.Predictor`, which owns audio decoding, resampling, the mel pipeline of Table 3, model loading, and thresholding. Preprocessing drift between training and serving is therefore structurally impossible rather than procedurally discouraged. Parity was verified empirically: 6/6 exact prediction matches against the archived training-time evaluation and a maximum absolute probability difference of 0.00148 over a 20-clip audit.

7.2 Packaging and the x86 guard

- **Docker**: a `python:3.11-slim` image pinned to `--platform linux/amd64`, installing CPU-only torch wheels; `libsndfile1` and `ffmpeg` are the only system dependencies.
- **ARM refusal**: because the INT8 artifact is `fbgemm`-packed (Section 5.4), `predict.py` detects non-x86 hosts and *refuses* to run INT8 there — failing loudly instead of returning collapsed probabilities — and directs the user to `--model fp32` or the Docker container. The FP32 student runs correctly on any architecture.
- **SmartThings path**: the production target is ARM, so the deployable INT8 artifact must be re-quantized with `qnnpack` (config already provisioned: `configs/experiments/quantize.yaml` sets `backend: qnnpack`); this is scheduled future work.

7.3 Live demonstration: Telegram bot

The public bot (`@kitpribot`) accepts voice messages and audio files (`.wav`/`.mp3`/`.ogg`/`.m4a`/`.flac`), runs the FP32 student at $\tau = 0.44$ through the shared `Predictor`, and replies Cooking/Not-Cooking. It uses `python-telegram-bot` v22 with *long polling*: the process makes outbound HTTPS calls to the Telegram API and requires **no public URL, webhook, TLS certificate, or open inbound port** — a property that makes the hosting architecture below essentially configuration-free.

Operational hygiene worth recording: the bot token is provided exclusively via environment variable (never argv, which is visible in `ps`); `httpx` request logging is silenced because Telegram API URLs embed the token; and exactly *one* poller may consume a token at a time — Telegram rejects concurrent `getUpdates` consumers with HTTP 409.

7.4 Always-on cloud hosting at $0

The bot initially ran on a developer laptop, making demo availability hostage to that laptop's lid. It now runs 24/7 on an **Oracle Cloud Infrastructure Always-Free** VM (Table 6), surviving crashes, reboots, and developer absence, at zero recurring cost.

Shape / region	VM.Standard.E2.1.Micro (1 OCPU x86, 954 MB RAM), ap-hyderabad-1
OS / runtime	Ubuntu 24.04 LTS · Python 3.12 venv · `torch` 2.13 (CPU wheels)
Memory strategy	2 GB swapfile provisioned before dependency install (1 GB RAM alone is insufficient for torch import + model load headroom)
Supervision	systemd unit `kitpri-bot.service`: `Restart=always`, `RestartSec=5`, enabled at boot, journald logging
Shutdown quirk	`TimeoutStopSec=15`: PTB's graceful shutdown can hang on SIGTERM; systemd escalates to SIGKILL, preventing zombie pollers
Secrets	token in `.env` (mode 600), delivered by `scp`, injected via systemd `EnvironmentFile`; never in the repository or logs
Network exposure	**none inbound** — long polling is outbound-only
Steady-state RAM	≈674 MB used including OS; model load ≈7 s on 1 core
Cost	$0 (Always Free tier, no time limit)

Table 6: Production hosting of the demonstration bot.

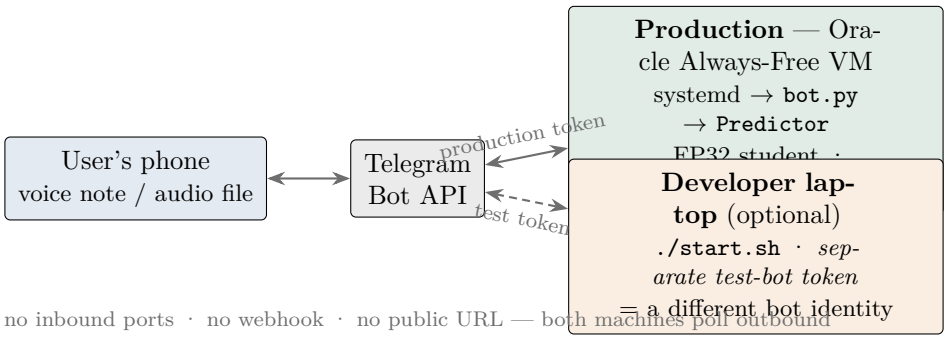

Figure 5: Serving topology. One token = one bot identity, and Telegram routes updates to the *newest* poller — so production and local testing use disjoint tokens, enforced by the launcher's identity guard (Section 7.5).

The deployment is reproducible from two committed artifacts: a step-by-step guide (`telegram_bot/DEPLOY.m`) and the systemd unit (`telegram_bot/kitpri-bot.service`).

7.5 Operational safeguards and developer ergonomics

The token-identity semantics above create one dangerous failure mode: because Telegram always transfers polling to the *newest* `getUpdates` consumer, a developer starting the production token on a laptop would *silently hijack* the live bot — the cloud instance receives the `409 Conflict` errors while user traffic drains to the laptop. This exact incident occurred once during development and motivated the following layered controls in the launcher (`start.sh` / `stop.sh`):

1. **Production-identity guard.** Before starting, the launcher resolves the configured token's identity via the Bot API (`getMe`) and *refuses* to start the production identity on a non-production machine. A deliberate override (`KITPRI_FORCE_PROD=1`) exists for planned failover, with instructions to stop the cloud service first.
2. **Conflict stand-down.** If any same-token clash slips through, the launcher detects Telegram's `Conflict` response in the process log within seconds, terminates the local instance, and explains the remedy.
3. **Locality by construction.** `stop.sh` terminates every bot process on the local machine (including manually started strays) and *cannot* affect the VM — production is only ever managed through an explicit `ssh …systemctl` command.
4. **Zero-setup onboarding.** The launcher bootstraps the entire environment (venv, CPU-only torch, dependencies, package) on first run. With no token configured it offers an interactive choice: a guided in-terminal @BotFather walkthrough that provisions a personal *test* bot, or an environment-only path for model development that prints the exact recipe for shipping a new checkpoint to the production bot.
5. **Config-driven model swap.** A new checkpoint is deployed by two `.env` lines (`KITPRI_BOT_CKPT`, `KITPRI_BOT_THRESHOLD`) honored identically under the local launcher and systemd; the `Predictor` auto-detects TorchScript vs. state-dict formats and fingerprints the architecture from checkpoint keys, so no code changes are involved.

8 Limitations and Future Work

Reported openly rather than omitted:

1. **Overlapped audio dominates the error budget.** Type-C clips (17% of data, SNR down to -5 dB) score 39–50% accuracy versus 75–78% on clean clips. This difficulty is intentional — it is the deployment reality the model must eventually master — but it is not yet mastered.
2. **Unstructured negatives.** The Not-Cooking pool is a single heterogeneous 1,160-clip set; curating it into subtypes (as was done for Cooking) would likely sharpen the boundary.
3. **The teacher is data-limited.** Val F1 peaks at epoch 1 (Figure 2); more diverse data and stronger augmentation — not a larger model — are the indicated remedies.
4. **INT8 threshold not independently re-tuned.** $\tau^* = 0.44$ was selected on FP32 validation probabilities. Quantization shifts calibration and coarsens the probability grid ($\approx$5 points near the boundary), so an INT8-specific sweep is outstanding.
5. **Recall below production bar.** The FP32 student misses $\approx$28% of cooking events at $\tau = 0.44$; INT8 misses $\approx$37% at 0.50. This milestone demonstrates a working end-to-end pipeline, not a shippable detector.
6. **INT8 artifact is x86-only.** `fbgemm`-packed; the ARM (SmartThings) target requires `qnnpack` re-quantization or quantization-aware training. The refusal guard prevents silent failure meanwhile.

7. **Licensing.** Negatives derive from ESC-50 (CC BY-NC 3.0); commercial deployment requires re-sourcing negative audio.

Future work, in priority order: `qnnpack` re-quantization for the ARM target; INT8-specific threshold re-sweep; recall-focused data expansion (more type-C-like training signal, curated negative subtypes); quantization-aware training if PTQ remains the bottleneck.

9 Conclusion

KitPri v4 demonstrates the full arc from research-grade transformer to edge-deployable artifact: a 329 MiB AST teacher distilled and quantized into a 2.80 MB TorchScript model — $> 118\times$ smaller, $2.25\times$ faster — while retaining 85% of the teacher's test F1 (INT8 at the default threshold) and 90% in FP32 form with governed threshold tuning. Equal engineering weight was placed on what surrounds the model: a deliberately hard synthetic-mixture dataset, one shared preprocessing/serving code path with verified parity, loud failure on unsupported hardware, and a zero-cost, always-on public demonstration under systemd supervision on Oracle's free tier. The limitations register makes the remaining distance to production explicit — ARM re-quantization, recall recovery, and negative-class curation — and each item is scoped against a config or script already provisioned in the repository.

References

[1] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio Spectrogram Transformer," *Proc. Interspeech*, 2021.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *Proc. CVPR*, 2018.

[3] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *NIPS Deep Learning Workshop*, 2015. arXiv:1503.02531.

[4] K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," *Proc. ACM Multimedia*, 2015. Licensed CC BY-NC 3.0.

[5] PyTorch Team, "Quantization — PyTorch documentation," https://pytorch.org/docs/stable/quantization.html.

[6] R. Wightman, "PyTorch Image Models (timm)," https://github.com/huggingface/pytorch-image-models.